LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL VI
PACKAGE & INHERITANCE

Disusun Oleh:

Alwisyah Putra 105224052

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

DAFTAR ISI

I. Pendahuluan.....3
II. Variabel..... 3
III. Constructor & Method..... 6
IV. Dokumentasi & Pembahasan Code.....9
V. Kesimpulan.....15
VI. Daftar Pustaka.....15

I. Pendahuluan

Take-Home Task Modul 6 kali ini disajikan studi kasus berbasis pemrograman berorientasi objek (OOP) yang mengimplementasikan sistem perhitungan gaji karyawan pada sebuah perusahaan. Studi kasus ini bertujuan untuk merancang dan membangun hierarki kelas yang terdiri dari superclass Karyawan serta dua subclass yaitu Developer dan Manajer, dengan menerapkan konsep encapsulation, inheritance, polymorphism, serta override method. Tantangan utama dari kasus ini terletak pada logika perhitungan gaji yang berbeda-beda di setiap jenis karyawan, seperti adanya bonus loyalitas berdasarkan masa kerja, penalti kinerja untuk rating rendah, tunjangan level dan bonus bug untuk developer, serta tunjangan manajerial dan bonus kinerja ekstra untuk manajer. Selain itu, pengelolaan objek polimorfik dalam ArrayList, validasi data, serta perhitungan statistik seperti total anggaran gaji, karyawan dengan gaji tertinggi, dan rata-rata rating kinerja menjadi tantangan tersendiri yang harus diselesaikan melalui kode program yang terstruktur dan efisien.

II. Variabel

1. Employee.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	name	String	Menyimpan nama karyawan.
2.	employeeID	String	Menyimpan ID unik karyawan.
3.	yearsOfJoin	int	Menyimpan tahun karyawan mulai bekerja.
4.	mainSalary	double	Menyimpan gaji pokok karyawan.
5.	performanceRating	double	Menyimpan rating kinerja karyawan (1.0–5.0).
6.	yearRightNow	int	Menyimpan tahun saat ini (2026) untuk menghitung masa kerja.
7.	yearsOfService	int	Menghitung lama kerja karyawan (tahun sekarang – tahun bergabung).
8.	loyaltyBonus	double	Menghitung bonus loyalitas = 5% × gaji pokok × masa kerja.

9.	temporarySalary	double	Menyimpan gaji sementara setelah ditambah bonus loyalitas, sebelum dipotong jika rating kinerja rendah.
10.	performanceRating	double	Parameter yang berisi nilai rating baru yang akan diset ke atribut performanceRating setelah divalidasi.

2. Manager.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	division	String	Menyimpan nama divisi yang dipimpin oleh manajer
2.	members	int	Menyimpan jumlah anggota tim yang dikelola oleh manajer
3.	baseSalary	double	Menyimpan hasil perhitungan gaji dari method calculateTotalSalary() milik class Employee (superclass)
4.	managerialAllowance	double	Menghitung tunjangan manajerial = 300.000 × jumlah anggota tim
5.	salaryAfterManagerial Allowance	double	Menyimpan gaji sementara setelah ditambah tunjangan manajerial, sebelum bonus kinerja (jika rating > 4.5)
6.	name	String	Nama karyawan (diteruskan ke superclass)
7.	employeeID	String	ID karyawan (diteruskan ke superclass)
8.	yearsOfJoin	int	Tahun bergabung (diteruskan ke superclass)
9.	mainSalary	double	Gaji pokok (diteruskan ke superclass)
10.	performanceRating	double	Rating kinerja (diteruskan ke superclass)

3. Developer.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	employeeLevel	String	Menyimpan level Dev: "junior", "mid", atau "senior"
2.	totalBugsFixed	int	Menyimpan jumlah bug yang telah diperbaiki oleh developer
3.	name	String	Nama karyawan (diteruskan ke superclass)
4.	employeeID	String	ID karyawan (diteruskan ke superclass)
5.	yearsOfJoin	int	Tahun bergabung (diteruskan ke superclass)
6.	mainSalary	double	Gaji pokok (diteruskan ke superclass)
7.	performanceRating	double	Rating kinerja (diteruskan ke superclass)
8.	employeeLevel	String	Level developer (disimpan ke atribut this.employeeLevel)

4. App.java

No.	Nama Variabel	Tipe Data	Fungsi
1.	employees	ArrayList<Employee>	Menyimpan daftar semua karyawan (objek Employee, Developer, dan Manager) yang akan diproses
2.	totalBudget	double	Mengakumulasi total gaji seluruh karyawan (anggaran perusahaan)
3.	totalRating	double	Mengakumulasi total rating kinerja seluruh karyawan (untuk menghitung rata-rata nanti)
4.	highestRatedEmployee	String	Menyimpan nama karyawan dengan gaji tertinggi (salah ngasih nama :'))

5.	highestSalary	double	Menyimpan nilai gaji tertinggi yang ditemukan
4.	emp	Employee	Variabel iterasi yang merepresentasikan satu karyawan dalam setiap putaran loop
5.	totalSalary	double	Menyimpan hasil calculateTotalSalary() untuk karyawan yang sedang diproses

III. Constructor & Method

1. Employee.java

No.	Nama Method	Jenis	Fungsi
1.	Employee(...)	Constructor	Menginisialisasi objek Employee dengan nama, ID, tahun bergabung, gaji pokok, dan rating kinerja. Melakukan validasi rating (1.0-5.0)
2.	getEmployeeID( )	instance method	Mengembalikan ID karyawan
3.	getYearsOfJoin( )		Mengembalikan tahun bergabung karyawan
4.	getMainSalary()		Mengembalikan gaji pokok karyawan
5.	getPerformanceRating()		Mengembalikan rating kinerja karyawan
6.	setEmployeeID(String)		Mengubah ID karyawan
7.	setYearsOfJoin(int)		Mengubah tahun

			bergabung karyawan
8.	setMainSalary(double)		Mengubah gaji pokok karyawan
9.	setPerformanceRating(double)		Mengubah rating kinerja dengan validasi (1.0-5.0)
10.	calculateTotalSalary()	Prosedural	Menghitung total gaji
11.	displayEmployeeInfo()	Instance Method	Menampilkan semua informasi karyawan ke terminal

2. Manager.java

No.	Nama Method	Jenis	Fungsi
1.	Manager ()	constructor	Menginisialisasi objek Manager dengan memanggil constructor superclass Employee, lalu menambahkan atribut division dan members
2.	getDivision()	Instance Method	Mengembalikan nama divisi yang dipimpin
3.	getMembers()		Mengembalikan jumlah anggota tim
4.	setDivision(String)		Mengubah nama divisi
5.	setMembers(int)		Mengubah jumlah anggota tim
6.	calculateTotalSalary()	Override Method	Menghitung total gaji Manager

7.	displayEmployeeInfo()	Override Method	Menampilkan informasi Manager

3. Developer.java

No.	Nama Method	Jenis	Fungsi
1.	Developer ()	Constructor	Menginisialisasi objek Developer dengan memanggil constructor superclass Employee, lalu menambahkan atribut employeeLevel dan totalBugsFixed
2.	getEmployeeLevel()	Instance Method	Mengembalikan level developer (junior/mid/senior)
3.	getTotalBugsFixed()		Mengembalikan jumlah bug yang telah diperbaiki
4.	setEmployeeLevel(String)		Mengubah level developer
5.	setTotalBugsFixed(int)		Mengubah jumlah bug yang diperbaiki
6.	calculateTotalSalary()	Override Method	Menghitung total gaji Developer

4. App.java

No.	Nama Method	Jenis	Fungsi
1.	main(String[])	Static Main Method	Entry point program. Membuat objek karyawan, menghitung total anggaran, rata-rata rating, dan

			mencari karyawan dengan gaji tertinggi

IV. Dokumentasi & Pembahasan Code

1. Employee.java

```java
package Perusahaan;

public class Employee {
    private String name;
    private String employeeID;
    private int yearsOfJoin;
    private double mainSalary;
    private double performanceRating;

    public Employee(String name, String employeeID, int yearsOfJoin, double mainSalary, double performanceRating) {
        this.name = name;
        this.employeeID = employeeID;
        this.yearsOfJoin = yearsOfJoin;
        this.mainSalary = mainSalary;
        this.performanceRating = performanceRating;

        if (performanceRating >= 1.0 && performanceRating <= 5.0) {
            this.performanceRating = performanceRating;
        } else {
            this.performanceRating = 3.0;
        }
    }

    public String getName() {
        return name;
    }

    public String getEmployeeID() {
        return employeeID;
    }
    public int getYearsOfJoin() {
        return yearsOfJoin;
    }
    public double getMainSalary() {
        return mainSalary;
    }
    public double getPerformanceRating() {
        return performanceRating;
    }

    public void setEmployeeID(String employeeID) {
        this.employeeID = employeeID;
    }
    public void setYearsOfJoin(int yearsOfJoin) {
        this.yearsOfJoin = yearsOfJoin;
    }
    public void setMainSalary(double mainSalary) {
        this.mainSalary = mainSalary;
    }
    public void setPerformanceRating(double performanceRating) {
        if (performanceRating >= 1.0 && performanceRating <= 5.0) {
            this.performanceRating = performanceRating;
        } else {
            this.performanceRating = 3.0;
        }
    }

    public double calculateTotalSalary() {
        int yearRightNow = 2026;
        int yearsOfService = yearRightNow - yearsOfJoin;

        double loyalityBonus = mainSalary * 0.05 * yearsOfService;
        double temporarySalary = mainSalary + loyalityBonus;

        if (performanceRating < 2.5) {
            temporarySalary *= 0.9;
        }

        return temporarySalary;
    }

    public void displayEmployeeInfo() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + employeeID);
        System.out.println("Years of Join: " + yearsOfJoin);
        System.out.println("Main Salary: " + mainSalary);
        System.out.println("Performance Rating: " + performanceRating);
        System.out.println("Total Salary: " + calculateTotalSalary());
    }
}
```

Pada file ini mendefinisikan kelas Employee di dalam package Perusahaan yang berfungsi sebagai kelas induk bagi kelas Developer dan Manager. Kelas Employee memiliki lima atribut privat yaitu name untuk menyimpan nama karyawan, employeeID untuk menyimpan kode identitas karyawan, yearsOfJoin untuk menyimpan tahun masuk kerja, mainSalary untuk menyimpan gaji pokok, dan performanceRating untuk menyimpan rating kinerja karyawan. Konstruktor kelas Employee menerima lima parameter dan menginisialisasi kelima atribut tersebut. Di dalam konstruktor, terdapat pengecekan. jika nilai performanceRating yang dimasukkan berada di antara 1.0 sampai dengan 5.0, maka nilai tersebut digunakan. namun jika tidak (kurang dari 1.0 atau lebih dari 5.0), maka performanceRating secara otomatis diatur menjadi 3.0. Kelas ini menyediakan metode getter untuk mengambil nilai dari keempat atribut (getEmployeeID, getYearsOfJoin, getMainSalary, getPerformanceRating) serta metode setter untuk mengubah nilai atribut (setEmployeeID, setYearsOfJoin, setMainSalary, setPerformanceRating). Pada metode setPerformanceRating, juga diterapkan aturan validasi yang sama seperti di konstruktor, yaitu jika rating di luar rentang 1.0-5.0 maka diatur menjadi 3.0. Method calculateTotalSalary() digunakan untuk menghitung total gaji karyawan.

Di dalam method ini, program mendeklarasikan variabel yearRightNow dengan nilai 2026, kemudian menghitung yearsOfService sebagai selisih antara tahun sekarang dengan tahun masuk karyawan. Selanjutnya program menghitung loyaltyBonus sebesar mainSalary * 0.05 * yearsOfService (bonus loyalitas 5% per tahun masa kerja), lalu menghitung temporarySalary sebagai mainSalary + loyaltyBonus. Setelah itu, program melakukan pengecekan: jika nilai performanceRating kurang dari 2.5, maka temporarySalary dikalikan dengan 0,9 (mengalami pemotongan 10%). Method ini kemudian mengembalikan nilai temporarySalary sebagai total gaji karyawan. Metode displayEmployeeInfo() digunakan untuk menampilkan informasi karyawan ke layar, yaitu mencetak nama karyawan, ID karyawan, tahun masuk, gaji pokok, rating kinerja, dan total gaji yang dihitung dengan memanggil metode calculateTotalSalary().

2. Manager.java

```java
package Perusahaan;

public class Manager extends Employee{
    private String divition;
    private int members;

    public Manager(String name, String employeeID, int yearsOfJoin, double mainSalary, double performanceRating, String divition, int members) {
        super(name, employeeID, yearsOfJoin, mainSalary, performanceRating);
        this.divition = divition;
        this.members = members;
    }

    public String getDivition() {
        return divition;
    }
    public int getMembers() {
        return members;
    }
    public void setDivition(String divition) {
        this.divition = divition;
    }
    public void setMembers(int members) {
        this.members = members;
    }

    @Override
    public double calculateTotalSalary() {
        double baseSalary = super.calculateTotalSalary();
        double managerialAllowance = 300.000 * members;
        double salaryAfterManagerialAllowance = baseSalary + managerialAllowance;

        if (getPerformanceRating()>4.5) {
            salaryAfterManagerialAllowance *= 1.15;
        }


        return salaryAfterManagerialAllowance;
    }

    @Override
    public void displayEmployeeInfo() {
        super.displayEmployeeInfo();
        System.out.println("Divisi: " + divition);
        System.out.println("Jumlah Anggota: " + members);
    }
}

```

Pada Class ini mendefinisikan kelas Manager yang merupakan turunan dari kelas Employee di dalam package Perusahaan. Kelas Manager memiliki dua atribut tambahan yaitu division untuk menyimpan nama divisi yang dipimpin oleh manager, dan members untuk menyimpan jumlah anggota tim yang berada di bawah manager tersebut. Konstruktor kelas Manager menerima tujuh parameter, di mana lima parameter pertama yaitu name, employeeID, yearsOfJoin, mainSalary, dan performanceRating diteruskan ke konstruktor kelas induk menggunakan kata kunci super, sedangkan dua parameter sisanya yaitu division dan members digunakan untuk menginisialisasi atribut milik kelas Manager sendiri. Kelas ini juga menyediakan metode getter (getDivision dan getMembers) serta setter (setDivision dan setMembers) untuk mengakses dan mengubah nilai dari kedua atribut tersebut.

Method calculateTotalSalary() ditandai dengan @Override yang menandakan bahwa method ini mengganti method dari kelas induk Employee. Di dalam method tersebut, program pertama-tama memanggil super.calculateTotalSalary() untuk mendapatkan gaji pokok dari kelas induk dan menyimpannya dalam

variabel baseSalary. Kemudian program menghitung managerialAllowance sebesar 300.000 * members, yaitu tunjangan manajerial yang besarnya 300.000 dikalikan dengan jumlah anggota tim. Setelah itu, program menghitung salaryAfterManagerialAllowance sebagai baseSalary + managerialAllowance. Selanjutnya program melakukan pengecekan: jika nilai getPerformanceRating() lebih besar dari 4.5, maka salaryAfterManagerialAllowance dikalikan dengan 1,15 (mendapat kenaikan 15%). Terakhir, metode ini mengembalikan nilai salaryAfterManagerialAllowance sebagai total gaji manager. Selain itu, kelas Manager juga mengoverride method displayEmployeeInfo(). Di dalam method ini, program pertama-tama memanggil super.displayEmployeeInfo() untuk menampilkan informasi dari kelas induk (seperti nama, ID, tahun masuk, gaji pokok, dan rating kinerja), kemudian mencetak tambahan informasi berupa divisi dan jumlah anggota tim yang dipimpin oleh manager tersebut.

3. Developer.java

```java
package Perusahaan;

public class Developer extends Employee {
    private String employeeLevel;
    private int totalBugsFixed;

    public Developer(String name, String employeeID, int yearsOfJoin, double mainSalary, double performanceRating, String employeeLevel, int totalBugsFixed) {
        super(name, employeeID, yearsOfJoin, mainSalary, performanceRating);
        this.employeeLevel = employeeLevel;
        this.totalBugsFixed = totalBugsFixed;
    }

    public String getEmployeeLevel() {
        return employeeLevel;
    }

    public int getTotalBugsFixed() {
        return totalBugsFixed;
    }

    public void setEmployeeLevel(String employeeLevel) {
        this.employeeLevel = employeeLevel;
    }
    public void setTotalBugsFixed(int totalBugsFixed) {
        this.totalBugsFixed = totalBugsFixed;
    }

    @Override
    public double calculateTotalSalary() {
        double baseSalary = super.calculateTotalSalary();
        double levelAllowance = 0.0;

        if (employeeLevel.equalsIgnoreCase("junior")) {
            levelAllowance = baseSalary + 0.0;
        }else if (employeeLevel.equalsIgnoreCase("mid")) {
            levelAllowance = baseSalary + 1.500000;
        } else if (employeeLevel.equalsIgnoreCase("senior")) {
            levelAllowance = baseSalary + 3.000000;
        }

        double salaryafterAllowance = baseSalary + levelAllowance;
        double bugBonus = 0;
        if (getPerformanceRating() >=3.0) {
            bugBonus = 50 * totalBugsFixed;
        }

        return salaryafterAllowance + bugBonus;
    }
}
```

Pada Class ini mendefinisikan kelas Developer yang merupakan turunan dari kelas Employee di dalam package Perusahaan. Kelas Developer memiliki dua atribut tambahan yaitu employeeLevel untuk menyimpan tingkat jabatan

developer dan totalBugsFixed untuk menyimpan jumlah bug yang telah diperbaiki. Konstruktor kelas Developer menerima delapan parameter, di mana lima parameter pertama yaitu name, employeeID, yearsOfJoining, mainSalary, dan performanceRating diteruskan ke konstruktor kelas induk menggunakan kata kunci super, sedangkan parameter employeeLevel dan totalBugsFixed digunakan untuk menginisialisasi atribut milik kelas Developer sendiri. Kelas ini juga menyediakan metode getter dan setter untuk mengakses dan mengubah nilai dari employeeLevel dan totalBugsFixed. Method calculateTotalSalary() ditandai dengan @Override yang menandakan bahwa metode ini mengganti metode dari kelas induk Employee.

Di dalam method tersebut, program memanggil super.calculateTotalSalary() untuk mendapatkan gaji pokok dari kelas induk, kemudian mendeklarasikan variabel levelAllowance dengan nilai awal 0.0. Program melakukan pengecekan terhadap nilai employeeLevel menggunakan metode equalsIgnoreCase untuk mengabaikan perbedaan huruf besar-kecil. Jika levelnya "junior", maka levelAllowance diisi dengan baseSalary + 0.0, jika levelnya "mid", maka levelAllowance diisi dengan baseSalary + 1.500000, jika levelnya "senior", maka levelAllowance diisi dengan baseSalary + 3.000000. Selanjutnya, program menghitung salaryAfterAllowance sebagai baseSalary + levelAllowance, kemudian mendeklarasikan variabel bugsBonus dengan nilai 0. Program memeriksa apakah nilai getPerformanceRating() lebih besar atau sama dengan 3.0, jika ya maka bugsBonus dihitung sebesar 50 * totalBugsFixed. Terakhir, metode ini mengembalikan nilai salaryAfterAllowance + bugsBonus sebagai total gaji developer.

4. App.java

```java
import Perusahaan.Employee;
import Perusahaan.Developer;
import Perusahaan.Manager;
import java.util.ArrayList;
public class App {
    public static void main(String[] args) throws Exception {
        ArrayList<Employee> employees = new ArrayList<>();

        employees.add(new Employee("K001", "Budi Santoso", 2020, 5000000, 2.0));
        employees.add(new Developer("D001", "Citra Dewi", 2018, 7000000, 4.0, "Senior", 20));
        employees.add(new Developer("D002", "Eko Prasetyo", 2024, 6000000, 2.8, "Mid", 15));
        employees.add(new Manager("M001", "Fitri Handayani", 2015, 10000000, 4.8, "IT", 10));
        employees.add(new Manager("M002", "Galih Putra", 2025, 8500000, 3.5, "HR", 3));

        double totalBudget = 0.0;
        double totalRating = 0.0;
        String highestRatedEmployee = "";
        double highestSalary = 0.0;

        for (Employee emp : employees) {
            emp.displayEmployeeInfo();
            double totalSalary = emp.calculateTotalSalary();

            System.out.println("Total Salary in this mount : " + totalSalary);
            System.out.println("----------------------------------");

            totalBudget += totalSalary;
            totalRating += emp.getPerformanceRating();

            if (totalSalary > highestSalary) {
                highestSalary = totalSalary;
                highestRatedEmployee = emp.getName();
            }
        }

        double averageRating = totalRating / employees.size();

        System.out.println("Total Budget for salary company: " + totalBudget);
        System.out.println("Average Performance Rating: " + averageRating);
        System.out.println("Highest Rated Employee: " + highestRatedEmployee);
        System.out.println("Highest Salary: " + highestSalary);
    }

}

```

Pada program main dimulai dengan mengimpor kelas-kelas yang diperlukan dari package Perusahaan yaitu Employee, Developer, dan Manager, serta ArrayList dari java.util. ketika membuat class atau program di folder terpisah pada project java maka untuk menggunakannya perlu diimport ke program main. kemudian mendeklarasikan kelas utama App dan metode main sebagai titik eksekusi. Di dalam method main, program membuat sebuah ArrayList bernama employees yang hanya dapat menyimpan objek bertipe Employee,

lalu saya menambahkan lima objek karyawan ke dalam daftar tersebut. satu objek Employee biasa (Budi Santoso), dua objek Developer (Citra Dewi dan Eko Prasetyo), serta dua objek Manager (Fitri Handayani dan Galah Putra), pada

method add disitu saya ketuker posisi value nama dan ID harusnya nama dulu baru ID sehingga nanti akan sedikit berbeda di bagian output. Selanjutnya program mendeklarasikan variabel totalBudget untuk menghitung total gaji seluruh karyawan, totalRating untuk menghitung rating kinerja, highestRatedEmployee untuk menyimpan nama karyawan dengan gaji tertinggi, dan highestSalary untuk menyimpan nilai gaji tertinggi. Program lalu memasuki perulangan for-each yang memproses setiap karyawan dalam ArrayList. untuk setiap karyawan, program memanggil method displayEmployeeInfo() yang bersifat polimorfik untuk menampilkan informasi sesuai tipe karyawan (Employee biasa, Developer, atau Manager), kemudian memanggil method calculateTotalSalary() yang juga polimorfik untuk menghitung total gaji karyawan tersebut (Employee biasa hanya gaji pokok, Developer mendapat bonus proyek, Manager mendapat tunjangan jabatan).

Setelah itu program mencetak total gaji karyawan tersebut, menambahkan gaji tersebut ke totalBudget, mengakumulasi rating kinerja ke totalRating, serta membandingkan total gaji karyawan saat ini dengan highestSalary; jika lebih besar, maka highestSalary diperbarui dan highestRatedEmployee diisi dengan nama karyawan tersebut. Setelah perulangan selesai, program menghitung rata-rata rating kinerja dengan membagi totalRating dengan jumlah karyawan, lalu mencetak laporan akhir yang terdiri dari total budget gaji perusahaan, rata-rata rating kinerja, nama karyawan dengan gaji tertinggi (meskipun variabel saya salah memberikan nama highestRatedEmployee), serta nilai gaji tertinggi itu sendiri.

V. Kesimpulan

dapat disimpulkan bahwa program ini berhasil menerapkan konsep Pemrograman Berorientasi Objek (OOP) meliputi encapsulation (dengan atribut private dan getter/setter), inheritance (kelas Employee sebagai superclass dan kelas Developer serta Manager sebagai subclass), serta polymorphism dan method overriding (metode calculateTotalSalary() dan displayEmployeeInfo() memiliki perilaku berbeda pada setiap subclass). Program mampu menghitung total gaji karyawan dengan logika berbeda-beda sesuai jenis karyawan (bonus loyalitas dan penalti kinerja untuk Employee biasa, tunjangan level serta bonus bug untuk Developer, tunjangan manajerial dan bonus kinerja untuk Manager), mengelola objek dalam ArrayList, serta menghasilkan laporan akhir berupa total anggaran gaji perusahaan, rata-rata rating kinerja, karyawan dengan gaji tertinggi, dan nilai gaji tertinggi tersebut. Seluruh alur program berjalan sesuai rancangan meskipun terdapat sedikit kekeliruan penamaan variabel (highestRatedEmployee seharusnya menyimpan karyawan dengan gaji tertinggi, bukan rating tertinggi) serta tertukarnya urutan parameter pada saat penambahan objek Employee. Secara keseluruhan, program ini telah memenuhi tujuan praktikum dalam mengimplementasikan konsep perulangan, percabangan, dan OOP secara terstruktur.

VI. Daftar Pustaka

Timotheus Ferdinand Tjondrojo and Yan Stephen Christian Immanuel Hutagalung (2026a) **Modul 4 - The Core of OOP.** Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

Timotheus Ferdinand Tjondrojo and Yan Stephen Christian Immanuel Hutagalung (2026b) **Modul 5 - Encapsulation.** Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

Timotheus Ferdinand Tjondrojo and Yan Stephen Christian Immanuel Hutagalung (2026b) **Modul 6 - Package & Inheritance.** Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.

THT Modul 6 (2026) Take Home Task Modul 4 dan 5, Universitas Pertamina.